



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

PHƯƠNG PHÁP PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG VỚI UML

BỘ MÔN CÔNG NGHỆ PHẦN MỀM



Giới thiệu nội dung học phần

Bài 1: Giới thiệu quy trình ICONIX Process

Bài 2: Mô hình hóa miền (Domain modeling)

Bài 3: Mô hình hóa Use Case (Use Case Modeling)

Bài 4: Requirements Review – Kiểm duyệt yêu cầu ➔ **Kiểm tra TX1**

Bài 5: Robustness Analysis — Cầu nối từ Use Case đến Thiết kế

Bài 6: Preliminary Design Review — Kiểm duyệt thiết kế sơ bộ ➔ **Kiểm tra TX 2**

Bài 7: Technical Architecture — Kiến trúc kỹ thuật

Bài 8: Sequence Diagrams

Bài 9: Critical Design Review (CDR) ➔ **Kiểm tra TX3**



TẦM QUAN TRỌNG CỦA MÔN HỌC

Môn học này có quan trọng không → Có. Vì:

- Nó là tài liệu bắt buộc trong đề tài, đồ án, dự án...
- Nó là cơ sở để cho các lập trình viên viết code
- Nó là tài liệu để cho các mô hình AI hiện đại sinh mã nguồn, chương trình...



BÀI 5: ROBUSTNESS ANALYSIS

Về kiến thức

 Khái niệm Robustness Analysis

 Vị trí trong ICONIX Process

 Quy trình 6 bước

 Top 10 Guidelines

 Ví dụ Internet Bookstore

 Cập nhật Domain Model

Về kỹ năng

 Hiểu vai trò Robustness Analysis

 Nhận diện boundary–controller–entity

 Thực hiện quy trình 6 bước

 Vẽ sơ đồ cho use case cụ thể

 Phát hiện lỗi logic, thiếu sót

 Kết nối Use Case → Design



Bối cảnh thực tế

01

⚠ Khoảng
trống giữa
phân tích &
thiết kế

02

🥚 Use Case
chỉ mô tả hành
vi

03

💻 Thiết kế
cần cấu trúc &
logic

04

🔄 Nhảy
thẳng sang
code = rủi ro

05

🧱 ICONIX
đưa ra cầu nối



Vị trí trong ICONIX

 Nằm trong giai đoạn Phân tích–
Thiết kế sơ bộ

 Sau Domain Model & Use Case

 Trước Sequence Diagram

 Kiểm duyệt tại PDR checkpoint

 Đầu vào cho thiết kế chi tiết



Cấu trúc sơ đồ Robustness



3 loại đối tượng chính



Boundary – Controller – Entity



Mối quan hệ Noun–Verb–Noun

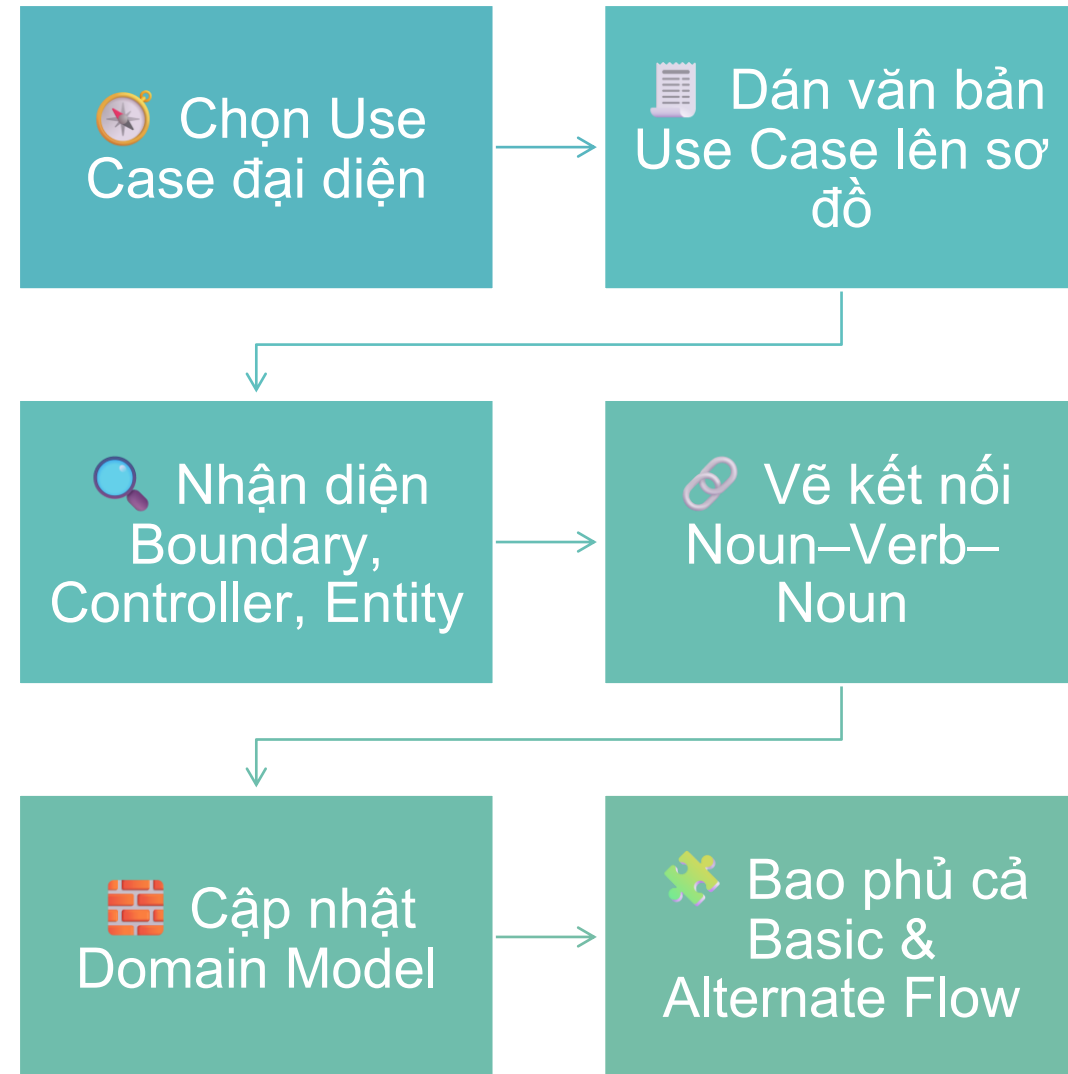


Actor $\leftrightarrow$ Boundary $\leftrightarrow$ Controller $\leftrightarrow$ Entity



Phát hiện lỗi logic sớm

Quy trình 6 bước tổng quát



Bước 1: Chọn Use Case tiêu biểu



Chọn Use Case quan trọng



Có luồng chính rõ ràng



Có dữ liệu và giao diện



Đại diện cho nhóm chức năng



Có thể mở rộng alternate

Bước 2: Dán Use Case lên sơ đồ



Copy
Basic &
Alternate Flow



Đặt cạnh
sơ đồ



Dễ đối
chiếu từng
câu



Phát hiện
câu mơ hồ



Giữ đồng
bộ văn bản &
hình

Bước 3: Nhận diện đối tượng



Noun → Entity hoặc Boundary



Verb → Controller



Mỗi lớp = 1 khái niệm



Dùng ký hiệu rõ ràng



Giới hạn trong phạm vi UC

Bước 4: Vẽ mối quan hệ Noun–Verb–Noun

-  Actor ↔ Boundary
-  Boundary ↔ Controller
-  Controller ↔ Entity
-  Không Boundary ↔ Entity trực tiếp
-  Mỗi đường = hành vi cụ thể

Bước 5: Cập nhật Domain Model



Thêm lớp mới phát hiện



Bổ sung quan hệ has-a / is-a



Ghi chú nguồn gốc từ UC



Giữ mô hình đồng nhất



Vòng lặp giữa phân tích & thiết kế



Bước 6: Bao phủ luồng chính & phụ



Basic Flow = đường chính



Alternate Flow = rẽ nhánh



Mỗi nhánh = nhánh logic riêng



Cùng sơ đồ hoặc tách riêng



Dùng màu để phân biệt

Top 10 Guidelines (Tổng quan)



Bộ quy tắc “vàng” ICONIX



Giúp kiểm tra sơ đồ nhanh



Áp dụng khi review nhóm



Phát hiện sai logic sớm



Chuẩn bị cho Sequence

#10: Paste Use Case Text

-  Dán văn bản ngay trên sơ đồ
-  Giữ ngữ cảnh
-  Thảo luận dễ dàng
-  Dễ đối chiếu dòng–dòng
-  Tăng độ chính xác

#9: One Controller per Use Case

⚙️ Mỗi Use Case = 1 controller

🧩 Tránh “God Controller”

🧱 Giữ phạm vi logic nhỏ

🧠 Dễ bảo trì, test

↺️↻️ Controller có thể gọi nhau



#8: Boundary không nói trực tiếp với Entity

 UI $\leftrightarrow$ Entity = sai nguyên tắc

 Mọi truy cập qua Controller

 Giữ tách biệt tầng giao diện

 Tăng tính ổn định hệ thống

 Dễ thay đổi UI không ảnh hưởng DB

#7: Actor chỉ nói với Boundary



Actor ↔ Boundary



Không Actor ↔ Controller



Không Actor ↔ Entity



Giữ đúng vai trò tương tác



Tránh shortcut sai thiết kế

#6: Mỗi danh từ = một lớp



Noun trong UC → Class



Cập nhật Domain Model



Tránh trùng tên khác nghĩa



Duy trì từ vựng miền



Giảm lỗi semantic drift



#5 đến #1 (Tóm tắt nhanh)



#5: Controller điều phối, không lưu trữ



#4: Sơ đồ bao phủ full Use Case



#3: Cập nhật Domain liên tục



#2: Không nhồi mọi thứ vào 1 sơ đồ



#1: Review nhóm, refine dần

Ví dụ: Login Use Case



Actor: Customer



Boundary: LoginScreen



Controller: LoginController



Entity: Customer



Alternate: Wrong password



Ví dụ: Show Book Details



Actor: Customer



Boundary: BestsellerList, BookDetailsScreen



Controller: ShowBookDetailsController



Entity: Book



Alternate: Book not found

Ví dụ: Write Review



Actor: Customer



Boundary: WriteReviewForm



Controller: WriteReviewController



Entity: Review, Book, Customer



Alternate: Invalid input

Lỗi thường gặp



Boundary
nói trực tiếp
với Entity



Một
Controller làm
quá nhiều



Thiếu
alternate
flows



Không
cập nhật
Domain



Không
review nhóm



Liên hệ với Domain Model

 Robustness phản ánh Domain sống

 Mỗi Entity trên sơ đồ = lớp Domain

 Controller tạo quan hệ động

 Bổ sung has-a / is-a mới

 Chuẩn bị cho Class Diagram

Từ Robustness → Sequence



Mỗi đường nối = lời gọi



Controller làm trung tâm



Boundary gửi, Entity phản hồi



Sinh Sequence cơ học



Giảm lỗi thiết kế

Kết luận & Ôn tập

-  Cầu nối “What” → “How”
-  Ba lớp: Boundary–Controller–Entity
-  Sáu bước thực hiện
-  Guidelines kiểm tra chất lượng
-  Liên hệ Use Case & Domain